could easily tie to D3. You may do that because you want to create images like we're doing here. Or you may use canvas because you can achieve greatly improved performance if you're dealing with large datasets. A number of online examples use canvas instead of SVG for D3 (especially with maps like the one at http://bl.ocks.org/mbostock/3783604, but also the implementation of a Voronoi diagram in canvas at http://bl.ocks.org/mbostock/6675193). But for our purposes, we don't need much code to create our image.

We'll use canvas to draw circles with numbers in them. We'll do this so we can have a set of images that we can use for our gallery. Your gallery probably has a set of images in a directory or called from an API, but because we don't have that here, we'll create them on the fly. At the same time, you'll get a sense of the functionality of the canvas API in regard to how it can be used alongside D3.

8.3.1 Drawing with canvas

The first thing we'll draw with canvas won't use much D3 code. What little it does use, such as `d3.select()` and `.node()`, could easily be replaced with native JavaScript. Later, when we start drawing many different images, and pass those images on to other elements, you'll see the kind of D3 functionality you've grown used to.

Listing 8.7 Canvas drawing code

```
d3.select("#traditional")
   .append("canvas")
   .attr("height", 500)
   .attr("width", 500);

var context = d3.select("canvas").node().getContext("2d");

context.strokeStyle = "black";
context.lineWidth = 2;
context.fillStyle = "red";
context.beginPath();
context.arc(250,250,200,0,2*Math.PI);
context.fill();
context.stroke();

context.textAlign = "center";
context.font="200px Georgia";
context.fillStyle = "black";
context.textAligh = "center";
context.fillText("1",250,250);
```

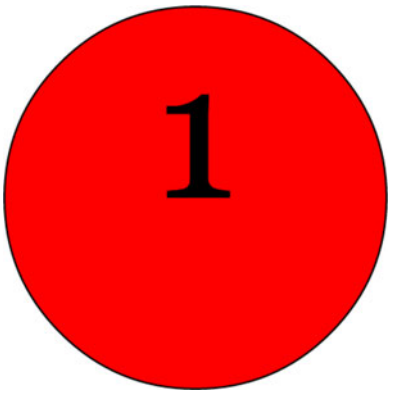


Figure 8.6 A circle and text drawn using HTML5 canvas

The result is the circle in figure 8.6. You'll notice a few important differences from the code we used earlier. First, we hardly use D3 in this example. We could easily have skipped it entirely by using the built-in selectors in core JavaScript. Second, we draw with canvas, not on an `<svg>`